

***nrlsmf* User's Guide**

COLLABORATORS			
	TITLE : <i>nrlsmf</i> User's Guide		
ACTION	NAME	DATE	SIGNATURE
WRITTEN BY		August 29, 2026	

REVISION HISTORY			
NUMBER	DATE	DESCRIPTION	NAME

Contents

1	Overview	1
2	Theory of Operation	1
3	Build Instructions	1
3.1	Elastic Multicast build	1
4	Usage	2
4.1	Usage Examples	2
4.2	Elastic Multicast Usage	3
5	Commands	3
5.1	GRE Tunnel Interfaces	13
5.1.1	Endpoint Address Discovery	13
5.1.2	Point-to-Point GRE	14
5.1.3	Multipoint GRE (mGRE)	14
5.1.3.1	Mode A — NHRP-resolved mGRE (dynamic hub-and-spoke)	14
5.1.3.2	Mode B — Statically-mapped mGRE (manual NBMA table)	16
5.1.3.3	Mode C — Multicast-underlay mGRE	17
5.1.3.4	Mixed underlay: multicast group plus unicast peers	18
5.1.4	External (metadata) GRE	19
5.1.5	Where Each Mode Fits	19
5.1.6	GRE Tunnel Commands	20
5.2	Virtual Interface Commands	20
5.3	VRF and FRR Integration	21
5.4	JSON Configuration	21
5.5	The "smfPkt" Command Format	22
6	Run-Time Remote Control	23
7	Current Limitations	24
8	Future Plans	25

List of Tables

1	Operating Mode Commands	4
2	Interface (and Interface Group) Commands	7
3	Elastic Multicast Commands	9
4	Gateway Commands	12
5	GRE / mGRE modes	19
6	GRE Tunnel Commands	20
7	Virtual Interface Commands	21
8	VRF Commands	21
9	Configuration Commands	21
10	Remote-only Commands	22

Abstract



The NRL Simplified Multicast Forwarding (nrlsmf) project includes software for a user-space IP Multicast forwarding engine. This software was developed by the **Naval Research Laboratory** (NRL) PROToCol Engineering Advanced Networking (**PRO-TEAN**) Research Group. The goal of this effort is to provide an implementation of experimental techniques for robust, efficient distribution of IP Multicast packets in dynamic, wireless networks such as Mobile Ad-hoc Networks (MANETs).

A PDF version is available for download: [nrlsmf User's Guide \(PDF\)](#).

1 Overview

The nrlsmf application can be run as a stand-alone application capable of providing "classic" flooding of broadcast and multicast traffic for a specified network interface or can be used in conjunction with a controlling program to perform more sophisticated multicast forwarding. An interprocess communication "remote control" interface is provided so that a compatible program (e.g. [nrlolsrd](#)) may issue run-time commands to nrlsmf to dynamically control the multicast forwarding process. Both IPv4 and IPv6 operation are supported. Versions of nrlsmf can be built for the following operating systems: Linux, MacOS, BSD, Win32, and WinCE. The code base also supports working simulation models of SMF in the ns-2 and OPNET discrete event simulation environments. In the future it is hoped that the SMF functionality might be included as part of operating system kernels.

2 Theory of Operation

The nrlsmf program acts as a user-space packet forwarding engine by promiscuously "sniffing" packets specified interfaces or intercepting packets using operating system firewall facilities and then retransmitting packets (using its local network interface MAC address) according to set forwarding rules. Packets can be received and possibly forwarded on a single, specified network interface for operation within a "routing area" corresponding to that single (generally wireless) interface. And, nrlsmf also provides for packet reception and forwarding across multiple interfaces to allow for configurable gateway operation and/or multicast distribution across a set of interfaces. Only incoming (non-locally generated), non-link-local IP Multicast packets with time-to-live (TTL) greater than one are considered for forwarding. Duplicate packet detection is an important facet of wireless network multicast forwarding since packets often must be forwarded on the same interface as they are received. Thus, neighbors' subsequent retransmission of forwarded packets will be "heard" and the local forwarding engine must discriminate between new packets and previously-forwarded packets to avoid unnecessary retransmission.

3 Build Instructions

The **nrlsmf** source code is available on Github at <https://github.com/USNavalResearchLaboratory/nrlsmf>. Note that **nrlsmf** depends upon NRL Protolib which is included as a submodule within the **nrlsmf** repository. Use the follow command to clone the **nrlsmf** repository with the Protolib submodule included:

```
git clone --recurse-submodules https://github.com/USNavalResearchLaboratory/nrlsmf ←  
.git
```

Then use the following commands to change to the "makefiles" directory and build with the appropriate Makefile.<system> for your operating environment. Note that build files for Windows (with some additional dependencies) and Android are available and this document will be updated to provide instructions for those systems. The following example is for Linux to build "legacy" **nrlsmf** without Elastic Multicast support.

```
cd nrlsmf/makefiles  
make -f Makefile.linux nrlsmf
```

3.1 Elastic Multicast build

Currently a separate build command is used to build **nrlsmf** with the Elastic Multicast routing capability. Note this separate build process will be eliminated in the near future and Elastic Multicast routing will be a standard feature of the regular **nrlsmf** build. Since Elastic Multicast and some related capabilities were developed as experimental "add-ons" to SMF, the separate build command was defined for initial work. To build **nrlsmf** with the Elastic Multicast features included, use:

```
cd nrlsmf/makefiles  
make -f Makefile.linux elastic
```

This will build an **nrlsmf** executable with the additional features and command available.

4 Usage

The nrlsmf program provides a command-line syntax with which it can be launched. Many of the same commands (see [Commands](#)) available via the command-line may be applied during run-time using the "remote control" interprocess control interface (see [Remote Control Interface](#)). To launch nrlsmf, use the following command-line syntax:

```
nrlsmf [version] [ipv6] [with-frr]
      [firewallForward {on|off}] [firewallCapture {on|off}]
      [add [<group>,] {cf|smpr|ecds|push|rpsh|merge|rmerge|tunnel}, <ifaceList>]
      [remove {<group> | [<group>,] <ifaceList>}]
      [elastic <group>] [advertise]
      [cf <ifaceList>] [smpr <ifaceList>] [ecds <ifaceList>]
      [push <srcIface>, <dstIfaceList>] [rpsh <srcIface>, <dstIfaceList>]
      [merge <ifaceList>] [rmerge <ifaceList>] [tunnel <ifaceList>]
      [forward {on|off}] [relay {on|off}] [delayoff <value>]
      [device <vifName>, <ifaceName> [/ {t|r|d}] [, <addr>...]]
      [map <iface>, <localAddr> [, <remoteAddr>]]
      [ujoin <groupAddr>, <iface>] [uleave <groupAddr>, <iface>]
      [vrf <vrf-name>, [<vrf-id>], <ifaceList>]
      [load <configFile>] [save <configFile>]
      [rate [<iface>,] <bits/sec>] [queue [<iface>,] <limit>]
      [unicast <ifaceGroupName>]
      [dscpCapture <dscpValue>, <dscpValueList>]
      [dscpRelease <dscpValue>, <dscpValueList>]
      [ihash <algorithm>] [hash <algorithm>]
      [idpd {on | off}] [window {on | off}]
      [instance <instanceName>] [smfServer <serverName>]
      [resequence {on|off}] [ttl <value>] [boost {on|off}]
      [debug <debugLevel>] [log <logFile>]
```

Run "nrlsmf -help" for the complete list of supported commands.

To send a command to an already-running instance:

```
nrlsmf --cli [-i <instanceName>] -c "<command>" [-c "<command>" ...]
```

4.1 Usage Examples

To run nrlsmf to modify outbound, locally-generated multicast packets for duplicate packet detection *and* to use "classical flooding" (CF) forwarding rules for packets received/forwarded on MANET interface "eth1":

```
nrlsmf resequence on cf eth1
```

Note that a MANET SMF forwarding rule like "cf" (or the others including "smpr", "ecds", etc) can be applied on a single interface or a group of interfaces. A single instance of nrlsmf will distribute traffic among all interfaces. If independent interfaces (or groups) are desired, then the user should invoke multiple instances of nrlsmf. For example, the syntaxes:

```
"nrlsmf cf eth0 cf eth1"
```

and

```
"nrlsmf cf eth0,eth1"
```

are equivalent. Both result in flooding of packets received on interfaces "eth0" and "eth1" to/among both of these interfaces. If two *separate* instances of nrlsmf are launched (one per interface) using the syntax

```
"nrlsmf cf eth0; nrlsmf cf eth1"
```

then the flooding is limited to each respective interface (i.e. packets received on "eth0" are possibly retransmitted only on "eth0"). Future versions of nrlsmf *_might_* provide the ability to set up multiple interface groupings with a single nrlsmf instance.

Using nrlsmf as a "gateway" between a typical network interface (e.g. Ethernet) and a MANET interface (e.g. 802.11 in ad hoc mode) can be accomplished using the "push" or "merge" commands (or their "r" (resequencing) variants). For example if it is

desired to "push" multicast traffic from a non-MANET interface (perhaps connected to a multicast router) "eth0" to a MANET interface "eth1", resequencing (or adding IPv6 SMF_DPD option header), run "classical flooding" on the MANET interface, and push multicast traffic originated on the MANET area to the non-MANET interface, the following syntax could be used:

```
nrlsmf rpush eth0,eth1 push eth1,eth0 cf eth1
```

Here the "rpush the0,eth1" command forces relay of traffic from eth0->eth1 and labels the packets (via the IPv4 ID field or IPv6 SMF_DPD option) for proper SMF duplicate packet detection. The "push eth1,eth0" command directs traffic from the MANET interface "eth1" to the non-MANET "eth0" interface, assuming that any traffic on MANET interface has already been "sequenced" (or tagged) appropriately for SMF duplicate packet detection. Note that nrlsmf maintains duplicate packet detection state for the "rpush" associate so that packet relayed from eth0 to eth1 are not reintroduced back to eth0 from eth1 even when other neighboring MANET nodes may retransmit those packets to support SMF data dissemination. Finally, the "cf eth1" command provides "classical flooding" on the presumably MANET eth1 interface (i.e. retransmitting received multicast packets so that other neighbors out-of-range of the heard transmission may receive the packets). Note that the "smprr" or "ecds" commands could be used here instead of "cf" for more efficient group data distribution if supporting neighborhood discovery protocol (NHDP) (or equivalent) information is operating and controlling nrlsmf state. Note that it is planned to integrate NHDP support into nrlsmf in a future version.

4.2 Elastic Multicast Usage

TBD

5 Commands

The nrlsmf program supports a variety of commands that can be invoked via command-line at startup or at run-time using the "remote control" interprocess communication interfaces that is provided. There are several categories of commands. These include:

1. "Operating Mode Commands" to control or enable general nrlsmf capabilities and features. Many of these commands can also be invoked at runtime via the remote-control interface set up by the nrlsmf **instance** <instanceName> command for dynamic changes to **nrlsmf** configuration and operation.
2. "Interface Commands" to enable SMF protocol operations on or among sets (aka Interface Groups) of network interfaces.
3. "Gateway Commands" to enable relaying of multicast to/from sets of interfaces. These commands can allow injection/extraction of multicast traffic for other interfaces to/from MANET interfaces to help create gateways between SMF and other multicast routing domains.
4. "GRE Tunnel Commands" for Linux GRE and mGRE interfaces (see [GRE Tunnel Interfaces](#)).
5. "Virtual Interface Commands" for SMF device and composite interface (CID) operation (see [Virtual Interface Commands](#)).
6. "VRF Commands" for virtual routing contexts and FRR integration (see [VRF and FRR Integration](#)).
7. "Configuration Commands" to load, save, and query JSON configuration (see [JSON Configuration](#)).
8. "Remote-only Commands" for dynamic, run-time control of specific aspects of nrlsmf operation. These commands are not intended for use via the command-line interface.

It should be noted that all of the commands listed here can be invoked via the run-time remote control interface (see [Remote Control Interface](#)). However, some commands, particularly the "Operating Mode Commands" are not intended to be invoked after initial startup. And, in general, the "Operating Mode Commands" SHOULD be invoked *before* other commands at startup. In some cases (firewallForward, etc), it is *required* that these commands precede "Interface" or "Gateway" commands.

As an alternative to increasingly complex nrlsmf command-lines, a JSON configuration file option is supported. To support initial transition to use of JSON configuration files, an nrlsmf **save** <fileName> command is available that will write a JSON configuration file upon nrlsmf exit using the state configured by the command-line and any runtime commands. The nrlsmf load <fileName> command is available to load such configuration files. The configuration file option is still a "work in progress"

and some command-line options are not yet fully reflected in the configuration file. Additionally, a change from the current 'bespoke' JSON format to one governed by a YANG data model is planned. A nice aspect of the JSON configuration is that its structure helps illustrate the relationship of various nrlsmf configuration elements including global attribute, interface groups, and per-interface attributes.

Table 1: Operating Mode Commands

load <filename.json>	Load an nrlsmf JSON configuration file using the specified <filename.json>.
save <filename.json>	Upon nrlsmf exit, save a JSON configuration file to the designated <filename.json>.
ipv6	If the <code>resequence</code> or various <code>firewall</code> interface commands are used, this enables setup of IPv6 instantiations of those options. This is an option since not all operating systems provide complete user space process access to the firewall chain for IPv6 traffic.
forward {on off}	Global control of all packet forwarding. If the "forward off" command is applied, all packet forwarding is disabled. This command may be invoked at run-time. (default = "on")
hash {MD5 CRC32 SHA1 NONE}	When I-DPD is disabled ("idpd off") and a hashing algorithm is enabled via this command (i.e., <algorithm> not equal to "NONE" (case-insensitive)), nrlsmf uses a hash-based duplicate packet detection (H-DPD per the SMF specification) approach to uniquely identify packets for forwarding. If this "hash" command is used in conjunction with "idpd on", I-DPD is performed for all packets except non-fragmented, non-IPSec IPv6 packets. For those excepted IPv6 packets, H-DPD is used and the SMF "Hash Assist Value" (HAV) is applied as needed when the "resequence on" option or "rpush/rmerge" operation is configured. Note that, depending upon the hash algorithm selected, this technique may be somewhat processing-intensive compared to the default approach that assumes an embedded unique identifier ("packetId") in packet headers. The "hash" and "ihash" commands are mutually exclusive and their use supercedes any prior hashing configuration. If I-DPD is disabled and no hash algorithm is specified, no duplicate packet detection occurs. (default = "NONE")
ihash {MD5 CRC32 SHA1 NONE}	This command is similar to the "hash" command described above in that a hashing <algorithm> type is specified for nrlsmf use. However, H-DPD operation is not invoked as a result of this command. Instead, the specified hash algorithm is used to calculate an "internal hash" value that is concatenated with the explicit I-DPD identifier for duplicate packet detection. This mode of operation enhances I-DPD operation, making it less vulnerable to denial-of-service attacks against SMF where spoofed packets with misleading packet identifiers may be generated by bad-behaving nodes. Note that nrlsmf "ihash" behavior when I-DPD is disabled (i.e., "idpd off") is undefined but will generally result in essentially H-DPD operation for forwarded flows. The "hash" and "ihash" commands are mutually exclusive and their use supercedes any prior hashing configuration. If I-DPD is disabled and no hash algorithm is specified, no duplicate packet detection occurs. (default = "NONE")

Table 1: (continued)

<code>idpd {on off}</code>	When enabled ("on"), the ID-based Duplicate Packet Detection (I-DPD per the SMF specification) is performed. Note that nrlsmf allows a hybrid of I-DPD and H-DPD to be used for packet identification via the "ihash" command. If I-DPD is disabled and no hash algorithm is specified, no duplicate packet detection occurs. (default = "on")
<code>filterDups {on off}</code>	When enabled ("on") duplicate packets received on nrlsmf device virtual interfaces are not written up to the kernel IP stack. This prevents local applications from receiving unnecessary duplicate packets. Note that this only applies to interfaces created with the nrlsmf device command where the nrlsmf interface I/O handling is "in path" and can block transfer of received packets unlike regular "pcap" I/O operation where nrlsmf is just seeing a copy of packets that are also provided to the kernel IP stack for receive processing. (default = "on")
<code>window {on off}</code>	When enabled ("on"), nrlsmf will use a windowed, sequence-number approach for duplicate packet detection. When disabled, nrlsmf uses a table-based approach that only requires unique packet identification without any requirement of ordered, sequenced packet identifiers. When "hash" mode is enabled, nrlsmf always uses the table-based approach to duplicate packet detection regardless of the "window" setting. Also when window is enabled, the hash type is automatically set to "NONE" and idpd operation is enabled. (default = "off")
<code>resequence {on off}</code>	Enables/disables interception and resequencing of locally-generated, outbound IP multicast packets. For IPv4 the packet ID field is modified and for IPv6 the SMF_DPD option header is applied. Packets that subject to IPsec are not modified. The resequence option leverages user space access to the operating system firewall chain to intercept outbound IP traffic.
<code>ttl <hopCount></code>	When <hopCount> is greater than ZERO, this command causes nrlsmf to intercept outbound, locally-generated IP multicast packets and set their TTL to the fixed hop-count specified by the <hopCount> value. If the <hopCount> value is less than or equal to ZERO, the TTL modification is disabled. This is provided for experimental use only and is NOT recommended for general use. (default = disabled).
<code>firewallForward {on off}</code>	When enabled, packets are forwarded through the host's IP stack, as opposed to the default method of generating raw MAC layer frames. With "firewallForward on", forwarded packets should be subject to IP firewall and queuing rules set up for applicable outbound interfaces. The default mechanism of raw MAC layer frame generation essentially bypasses the IP stack and any such configured IP policies or rules. When used, the "firewallForward" command <i>MUST</i> precede any "Interface" or "Gateway" commands on the command-line. (default = "off") [DEPRECATED]

Table 1: (continued)

<code>firewallCapture {on off}</code>	When enabled, packets are captured using operating system IP firewall capabilities instead of the default raw MAC layer frame capture. It should be note that when this is enabled, that packets are captured after any re-assembly of fragmented IP packets has occurred due to MTU limits. The default mechanism of capturing raw MAC layer frames will not properly forward any IP packets that have been fragmented in the current nrlsmf implementation. When used, the " <code>firewallCapture</code> " command <i>MUST</i> precede any "Interface" or "Gateway" commands on the command-line.(default = "off") [<i>DEPRECATED</i>]
<code>tap <tapInstanceName></code>	This command instructs nrlsmf to pass packets that would be forwarded to an external process instead of forwarding them. Note, however, that the external process may use the interprocess "smfPkt" command to return these packets back to the nrlsmf process for actual forwarding. The external process can use the "smfPkt" command and its format to send interprocess messages with packet content to nrlsmf instances identified by their <instanceName>. See the " smfPkt " command description for further information. This allows an external process to "tap" into the SMF forwarding plane and perform additional, custom filtering of forwarded packet flows. A simple, pass-through example application is included in the nrlsmf source code distribution in the file "tapExample.cpp".
<code>instance <instanceName></code>	Assigns a name to the running nrlsmf process. This name is used to name the ProtoPipe(MESSAGE) mechanism (Unix-domain socket on Unix, mailslot on Win32, registry-entered host-local UDP socket on WinCE) for interprocess communication that is monitored for "remote control" commands (including most of the command-line options listed in this table). (default = "nrlsmf")
<code>smfServer <processName></code>	Specify a compatible, external process that nrlsmf will inform of its status. (nrlsmf will signal that process with a "smfClientStart <instanceName>" message via interprocess communication). It is expected that such an external process may wish to have dynamic control of nrlsmf. This signaling allows the nrlsmf and external processes to be started independently in any order. (default external process name = "nrlolsr")
<code>unicast <ifaceGroupName></code>	Enables unicast flooding or "Elastic Unicast" routing operation for interface groups configured with the <code>elastic</code> option. "Elastic Unicast" routing is somewhat like an On-Demand Distance Vector MANET routing protocol.
<code>dscpCapture <dscpValue>, <dscpValueList></code>	Specify a list of DSCP values for unicast packets that will be intercepted by nrlsmf. This command assumes that a unicast command is also being used. [<i>DEPRECATED</i>]
<code>debug <debugLevel></code>	Sets the nrlsmf debug level. The higher the <debugLevel> value, the more verbose the debug output. (default = 0)
<code>log <logFile></code>	Redirects nrlsmf debug output to a file instead of <code>stderr</code> .
<code>boost {on off}</code>	Enables/disables "priority boost" (process priority) for the nrlsmf process. It may be useful to disable "priority boost" for debugging purposes. (default = 'on')
<code>with-frr</code>	Enables nrlsmf to run along FRR and pull configuration where appropriate, such as VRF info
<code>vrf <vrf-name>, [<vrf-id>,<ifaceList></code>	Associate list of interfaces with a VRF.

The "Interface Commands" apply to configuring specific, usually wireless, interfaces for SMF operation. As such, packets inbound on such interfaces may also be forwarded by nrlsmf back out the same interface as well as possibly other interfaces if so configured. The "resequence" command listed here should be used to ensure that packets generated are properly labeled using the IPv4 ID field or the IPv6 SMF_DPD header option for SMF duplicate packet detection.

Table 2: Interface (and Interface Group) Commands

<pre>add [<ifaceGroup>,<ifaceList></pre>	Creates a new interface group or adds interfaces to an existing one and configures the interface group for a specific relay algorithm (e.g., cf, smpr, ecds). Also, a "push", "rpush", "merge", or "rmerge" gateway interface group can also be configured with the 'add' command instead of an SMF relay algorithm. Note if the <ifaceGroup> name is omitted, an implicit group name based on the relay algorithm or given gateway command (e.g., 'cf', 'ecds', 'push', etc) is assumed. An "interface group" corresponds to a set of interfaces that are treated as a common flooding or Elastic Multicast routing domain. (Note that nrlsmf is currently limited to a single "elastic" interface group but will allow for multiple, separate "elastic" groups in the future.
<pre>remove <ifaceGroup>,<ifaceList></pre>	Removes either an entire <ifaceGroup> or the given <ifaceList> interfaces from a specific <group> or from all groups if the <group> name is omitted. The <group> name should be that of an explicit group name previous created with the add command or the implicit relay algorithm or gateway mode.
<pre>layered <ifaceName></pre>	This per-interface attribute marks the given <ifaceName> as an interface that is associated (layered) with a subnetwork that provides its own organic multi-hop, multicast distribution as opposed to simple wireless interfaces (e.g. WiFi) that don't provide underlying multi-hop, multicast distribution. The effect of this attribute is that is a given packet is received upon such an interface, the duplicate packet detection table for that interface is updated so that same packet will not be forwarded out that interface since nrlsmf can assume (via the layered attribute) that the packet has already been fully distributed on that subnetwork as needed. This avoids "packet storms" where both an nrlsmf "overlay" and the underlay subnetwork would be duplicatively (or more) flooding/forwarding multicast packets over possibly multiple hops. This same attribute can also be applied to point-to-point interfaces or tunnel interfaces where forwarding a packet out the same interface upon which it was received has no (or negative) benefit.
<pre>cf <ifaceList></pre>	Enables classical flooding (CF) with duplicate packet detection among one or more listed interfaces. Note packets received on a given interface may be forwarded out the same interface as well as on other listed interfaces. The <ifaceList> is a comma-delimited (no spaces) list of interface names. This command is supported for backwards compatibility and its use is equivalent to using the add command to create an interface group names 'cf'. I.e.,:
<pre>smpr <ifaceList></pre>	Enables S-MPR relaying with duplicate packet detection among one or more listed interfaces. Note packets received on a given interface may be forwarded out the same interface as well as on other listed interfaces. The <ifaceList> is a comma-delimited (no spaces) list of interface names.

Table 2: (continued)

<code>ecds <ifaceList></code>	Enables E-CDS relaying with duplicate packet detection among one or more listed interfaces. Note packets received on a given interface may be forwarded out the same interface as well as on other listed interfaces. The <code><ifaceList></code> is a comma-delimited (no spaces) list of interface names.
<code>relay {on off}</code>	This command controls the nrlsmf status as a relay for interfaces that have E-CDS forwarding enabled. Note that S-MPR relay status is controlled by the selector MAC address list supplied to nrlsmf during run-time. (default = "off"). In the future, this command will be updated to apply to specific interface group(s). At the moment, it applies to all interface groups.
<code>allow [vrf, <srcVRF>, <dstVRF>, {all <addr1>[, <addr2>, ...]}</code>	Sets a forwarding policy to allow either specific Virtual Routing and Forwarding (VRF) groups with specific destination addresses or globally allow specific addresses to be flooded/routed by nrlsmf. Non-matching addresses will be ignored by forwarding within the VRF group (if 'vrf' is specified) or globally. An address keyword of 'all' can be used to wildcard (i.e., "allow all"). The complementary deny command provides a way to edit the overall policy to exclude specific addresses. The allow and deny commands can be used in combination to control the overall policy.
<code>deny [vrf, <srcVRF>, <dstVRF>, {all <addr1>[, <addr2>, ...]}</code>	Sets a forwarding policy to deny (block) destinations from forwarding within either specific Virtual Routing and Forwarding (VRF) groups or globally. Matching addresses will be ignored by forwarding within the VRF group (if 'vrf' is specified) or globally. An address keyword of 'all' can be used to wildcard (i.e., "deny all"). The complementary allow command provides a way to edit the overall policy to exclude specific addresses. The allow and deny commands can be used in combination to control the overall policy.
<code>device <vifName>, <ifaceName> [, <addrList ...>]</code>	This command instantiations a tun/tap virtual interface named <code><vifName></code> that is associated with the underlay physical interface identified by <code><ifaceName></code> . If the optional <code><addrList></code> is omitted, the default behavior is that addresses and routes assigned to <code><ifaceName></code> will be "harvested" and reassigned to the new <code><vifName></code> interface. Those addresses and routes are restored to the <code><ifaceName></code> interface when nrlsmf shuts down.
<code>cid <vifName>, <iface1>[/ {t r d}] [, <iface2>[/ {t r d}] [, <iface3>[/ {t r d}] , ...]</code>	The Composite Interface Device (cid) command can be used to add (or reconfigure) physical interface devices to an existing 'parent' virtual interface created by the nrlsmf device command. The listed interfaces act as "subchannels" to the parent interface. Packets received on receive-enabled ('r' flag) interfaces are passed up to the kernel IP stack via the <code><vifName></code> tap device. Packets transmitted to the <code><vifName></code> interface are currently "mirrored" (i.e., replicated) across transmit-enabled ('t' flag) sub-interfaces. The code base also supports round-robin distribution (but not currently exposed) and other experimental strategies (e.g., network coding) may be added in the future. The optional 't', 'r', and 'd' flags are available to designate sub-interfaces as transmit-only, receive-only, or be deleted (i.e., removed) respectively. If these flags are omitted, the default sub-interface behavior is to be used both for transmission and reception. This feature is primarily to support research and development activities where bonding of different (nominally wireless) interfaces can be useful for different multi-channel concepts. Note the given <code><vifName></code> MUST be an nrlsmf device that has already been created.

Table 2: (continued)

rate [<ifaceName>,<bits/sec>	This command sets a maximum transmission rate limit for the interface identified by <ifaceName>. The default behavior is unlimited rate. Note a <bits/sec> value of -1.0 can be used to indicate unlimited rate.
queue [<ifaceName>,<queueLimit>	This command enables an nrlsmf layer of queuing in addition to whatever queuing (e.g., Linux 'tc' traffic control) is enabled on the underlying physical interface. Note that <ifaceName> may correspond to a virtual interface created by the device command or other physical interface that has been assigned to an nrlsmf interface group. The intent of this option was to allow for experimental queuing and flow control mechanisms in conjunction with routing and forwarding but is limited by minimum interface queue depths and operation with the current use of AF_PACKET interfaces for packet I/O. Future updates to nrlsmf may utilize AF_XDP or PACKET_TX_RING on Linux to address that limitation. The <queueLimit> parameter indicates the number of packets the queue will buffer and a simple "drop tail" queuing discipline is observed.
ujoin <groupAddr>,<ifaceName>	This is for use with mGRE point-to-multipoint tunnel interfaces where IP multicast encapsulation is used on the underlay network. The ujoin (underlay join) command causes nrlsmf to join the designated IP multicast <groupAddr> on the given <ifaceName>. This is generally needed for GRE decapsulation to successfully work with multicast-underlay mGRE (the kernel tunnel remote is the group). On a wildcard-remote mGRE device the kernel does not demux GRE-in-multicast by outer destination; if that same group is also a mapped inject remote, ujoin additionally enables an underlay capture so nrlsmf can strip GRE and treat the inner packet as inbound on the overlay GRE interface. Future nrlsmf Elastic Multicast operation may support dynamic underlay group membership depending on the overlay multicast -> underlay multicast mapping scheme used. Meanwhile this command can be used as part of static configuration or dynamically at runtime by a controller as needed.
uleave <groupAddr>,<ifaceName>	This is the complement of the ujoin command and is used to leave underlay group memberships as needed.

The "Elastic Multicast Commands" are the set of commands that apply only to Elastic Multicast protocol operation supported by nrlsmf. Currently, Elastic Multicast operation is limited to a single nrlsmf Interface Group. In the future, separate Multicast Forwarding Information Bases (aka Mcast FIBs) will be supported on a per Interface Group basis. The nrlsmf code update plan includes ongoing separation of control plane functions (protocol signaling, configuration) and forwarding/data plane functions (packet reception/transmission, duplicate packet detection, flow detection and timeout) to streamline the code and its performance. Some aspects of Elastic Multicast design are still in progress informed by limited deployment and experimentation activities.

Table 3: Elastic Multicast Commands

Table 3: (continued)

elastic <ifaceGroup>	Enables Elastic Multicast routing instead of basic SMF flooding for the given <ifaceGroup>. IMPORTANT - note that nrslsmf is currently limited to providing proper Elastic Multicast routing on a single interface group. Future update will enable multiple, different interface groups to conduct independent Elastic Multicast routing. Thus, "interface groups" will act much like Virtual Routing and Forwarding (VRF) groups. Note that for basic SMF flooding, different interface groups can conduct forwarding independent of one another.
advertise	Set Elastic Multicast operation to use Elastic Multicast Advertisement (EM_ADV) messages to inform other network relays of active flows instead of the default token bucket limited forwarding of user data packets. However, note that upon initial detection of a flow, the first token bucket depth count of user data packets are forwarded/flooded per SMF to allow limited data transfer while initial EM_ADV propagation and any Elastic Multicast Acknowledgment (EM_ACK) messaging occurs to build the distribution tree/mesh without loss of user traffic. This also allows support of very low duty cycle multicast flows that would otherwise be penalized by initial forwarding tree/mesh formation. In the future the Elastic Multicast "advertise" attribute will be controllable on a per interface group basis.
etx <ifaceName>	Enable use of a path quality metric similar to Estimated Transmission Cost (ETX) on the interface specified by <ifaceName>. This is only useful when the elastic command is used to enable Elastic Multicast routing. If ETX is not available, Elastic Multicast routing uses a hop count metric by default. To support link quality measurement to inform the metric, a modified version of the IP_UMP (Upstream Multicast Packet) header extension is used. The modification includes setting a 16-bit sequence number in the reserved portion of that header extension that is used to measure per-hop packet loss on all packets forwarded. The ETX-like metric is accumulated along the forwarding path using this information and "downstream" relays pick the best "upstream" relays to acknowledge (using EM_ACK messages) to activate the best quality forwarding paths. The per-hop 'cost' of a hop with no packet loss is one with lossy hops adding additional 'cost' to build the additive path metric. Thus non-ETC interfaces are assumed lossless with a one-hop cost of one and mix of ETX and non-ETX interfaces can be used. Elastic Multicast will be extended with additional metric options in the future.

Table 3: (continued)

<pre> flow <srcAddr>[/<maskLen>]->] <dstAddr>[/<maskLen>] [, <protocol>[, <class>] </pre>	Administratively emplace a flow description that will be proactively advertised when Elastic Multicast advertise mode is enabled. Note a local interface name can be used in place of the <code><srcAddr></code> parameter and the IP address for the given interface will be used as the source interface in the advertisement. This enables nrlsmf to proactively build multicast trees independent of user data transmission. One experimental use case being pursued includes using this capability to build trees for use by hop-by-hop content distribution mechanisms. The nrlsmf code base has a notification function when previous and/or next hop information changes for a flow and the source->group information can be used to match the appropriate prev/next hops for content distribution. Other use cases can include configuration of preset flows for a prior known user data transmissions, including low duty cycle flows. Zero-value <code><maskLen></code> fields will wildcard addressing. An IP <code><protocol></code> value of 255 will wildcard that aspect of the flow description and a zero-value <code><class></code> does the same for its aspect.
<pre> join <srcAddr>[/<maskLen>]->] <dstAddr>[/<maskLen>] [, <protocol>[, <class>] </pre>	Administratively "join" a flow description that will prompt Elastic Multicast routing to acknowledge (via EM_ACK) matching upstream flows. Nominally, this can simply be an IP multicast <code><dstAddr></code> group or a more specific (or less specific with the <code><maskLen></code> option) flow classifier, even including unicast if nrlsmf unicast operation is enabled. The same field specifications apply as for the flow command.
<pre> leave <srcAddr>[/<maskLen>]->] <dstAddr>[/<maskLen>] [, <protocol>[, <class>] </pre>	Administratively "leave" a flow previously set with the join command. This will cancel acknowledgment of matching flows for Elastic Multicast operation.
<pre> map <iface>,<localAddr> [, <remoteAddr>] </pre>	Maps tunnel endpoint address information for a GRE interface (or other ancillary iface->address associations). This enables Elastic Multicast operation to properly match inbound EM_ACK messages to the interface upon which the corresponding EM_ADV or forwarded packet was transmitted. Repeat the command with the same local address and a different remote to record additional remotes; overlay multicast forwarded onto a wildcard-remote mGRE interface is transmitted once to each mapped remote that is a unicast peer or an underlay multicast group. A remote of <code>dynamic</code> learns unicast remotes from the kernel neighbor table on <code><iface></code> and keeps them updated (NHRP or static NBMA). It does not apply to metadata/"external" GRE, which needs explicit unicast remotes for overlay-multicast inject. A remote of <code>0.0.0.0</code> is the kernel wildcard (INADDR_ANY), not a send destination. Typically this command is not needed for point-to-point GRE or multicast-underlay mGRE (the tunnel remote is already the group). Runtime mappings can be listed with "show tunnel": Local/Remote are underlay tunnel endpoints and IP is the local overlay address. C in Flags means the mapping was added with this command; otherwise it was learned from the kernel. GRE/mGRE neighbors are listed with "show tunnel neighbors": Neighbor IP is the peer overlay address and Remote is the peer underlay. Neighbor flags use C for configured plus the kernel NUD state: R reachable, S stale, D delay, P probe, I incomplete, F failed, N no-ARP, M permanent (for example CR is configured and reachable).

Table 3: (continued)

unmap <iface>, <localAddr> [, <remoteAddr>]	Deletes matching tunnel endpoint or interface->address associations previously set up with the map command. A remote of dynamic stops neighbor learning and removes learned remotes; explicit unicast map entries are left in place.
reliable <ifaceList>	Enable experimental reliable hop-by-hop forwarding on listed interfaces (adds UMP option to IPv4 packets).
utos <trafficClass>	Set the IP traffic class value to be ignored by reliable forwarding.
adaptive <group>	Enable Smart Routing (adaptive routing) for the named interface group.

The "Gateway Commands" apply to configuring nrlsmf to support a forced relay of packets to/from MANET/non-MANET interfaces. This can allow creation of gateways from infrastructure IP networks (e.g., the Internet) with IP Multicast routing support to dynamic MANET areas that rely upon SMF for group data distribution. The commands provided here are designed to have a great deal of flexibility, so some care must be taken to avoid loops with the modes (i.e., "rpush" or "rmerge" commands) that "resequence" (or tag) packet flows for duplicate packet detection. The nrlsmf application has some logic in its command-parsing to avoid some of these scenarios, but the flexibility allowed permits some configurations that may produce undesirable effects.

Table 4: Gateway Commands

push <srcIface>, <dstIfaceList>	Force relay of packets from <srcIface> to listed destination interfaces. The <dstIfaceList> is a comma-delimited (no spaces) list of interface names. Packet forwarding is subject to duplicate packet detection and TTL/hop limit constraints. Useful for setting up an SMF "gateway" to inject packets to/from a MANET SMF area.
rpush <srcIface>, <dstIfaceList>	Resequence (modify IPv4 ID field or add IPv6 SMF-DPD option header) and force relay of packets from <srcIface> to listed destination interfaces. The <dstIfaceList> is a comma-delimited (no spaces) list of interface names. IPSec marked packets are not modified. The potential for reverse relaying of packets is avoided by keeping DPD state on the <srcIface> using the resequenced identifiers. Useful for setting up an SMF "gateway" to inject packets from a non-MANET (e.g., infrastructure Internet) interface where sources may not be applying proper sequencing to support SMF duplicate packet detection. Note that the packet identifier "resequencing" (or "tagging") performed here is for "forwarded" (i.e. relayed) packets and is distinct from the "resequence" command that applies to locally-generated IP Multicast packets.
merge <ifaceList>	Force relay of packets from any listed interface to all other listed interfaces. The <ifaceList> is a comma-delimited (no spaces) list of interface names. Packet forwarding is subject to duplicate packet detection and TTL/hop-limit constraints. Useful for setting up an SMF "gateway" to bridge together different networks using essentially classical flooding among the interfaces, but not ever retransmitting the packet back to the inbound interface.

Table 4: (continued)

<pre>rmerge <ifaceList></pre>	Resequene and force relay of packets from any listed interface to all other listed interfaces. The <code><ifaceList></code> is a comma-delimited (no spaces) list of interface names. Note the potential for packet ping-pong due to reverse relaying from any MANET interfaces in the <code><ifaceList></code> . Thus, this command should be used with caution and forethought, or maybe never. Note that the packet identifier "resequencing" (or "tagging") performed here is for "forwarded" (i.e. relayed) packets and is distinct from the "resequence" command that applies to locally-generated IP Multicast packets.
-------------------------------------	--

5.1 GRE Tunnel Interfaces

nrlsmf supports operation on Linux Generic Routing Encapsulation (GRE) and point-to-multipoint (mGRE) interfaces. From a configuration standpoint, a GRE or mGRE interface is just another interface name that can be placed into an nrlsmf interface group alongside physical interfaces. Overlay unicast peer resolution (kernel `ip neigh`, NHRP, or a multicast underlay group) stays outside nrlsmf. Overlay *multicast inject* onto a wildcard-remote mGRE interface is the exception: nrlsmf transmits once per mapped remote (unicast peers and, optionally, an underlay multicast group). Overlay GRE interfaces are typically **layered** so a packet received on the tunnel is not flooded back out of it. The NHRP hub is the exception: it must replicate overlay multicast to the spokes. In every mode nrlsmf still floods decapsulated overlay multicast per its relay rules and keeps endpoint information for duplicate packet detection (DPD) and Elastic Multicast.

GRE interfaces are treated differently from Ethernet interfaces: packets received on a GRE interface may not include a usable Ethernet header, and tunnel endpoint addresses (local and remote) are used for forwarding and Elastic Multicast control traffic.

5.1.1 Endpoint Address Discovery

When a GRE interface is opened, nrlsmf reads the tunnel's local and remote endpoint addresses from the operating system. This is enough for point-to-point GRE and for multicast-underlay mGRE (the tunnel remote is already a multicast group): there is a single remote, so one transmission has a well-defined destination.

`map <iface>,<localAddr>[,<remoteAddr>]` records tunnel endpoint information in nrlsmf's interface-info table. Multiple **map** commands with the same local address and different remotes add multiple peers. That list is used in two ways:

- DPD / Elastic Multicast bookkeeping (all GRE modes).
- Overlay multicast inject onto a wildcard-remote (0.0.0.0) mGRE interface: the kernel has no single destination for overlay multicast, so nrlsmf transmits once per mapped remote that is a unicast peer or an underlay multicast group. Unicast remotes come from explicit `map <iface>,<local>,<peer>` and/or `map <iface>,<local>,dynamic` (kernel neighbor table). A mapped remote may also be an underlay multicast group; that is an explicit **map**, not learned by `dynamic`, and inbound GRE-in-multicast on that group needs **ujoin** as well. Overlay unicast on that same interface still uses the kernel neighbor table (`ip neigh`), not **map**. `map gre0,10.0.0.2,0.0.0.0` is allowed: it records the kernel wildcard remote (`INADDR_ANY`) for DPD / Elastic Multicast bookkeeping. It is not a send destination and not a synonym for `dynamic`. Opening a tunnel that already has `remote 0.0.0.0` stores the same mapping, so that command is usually redundant.

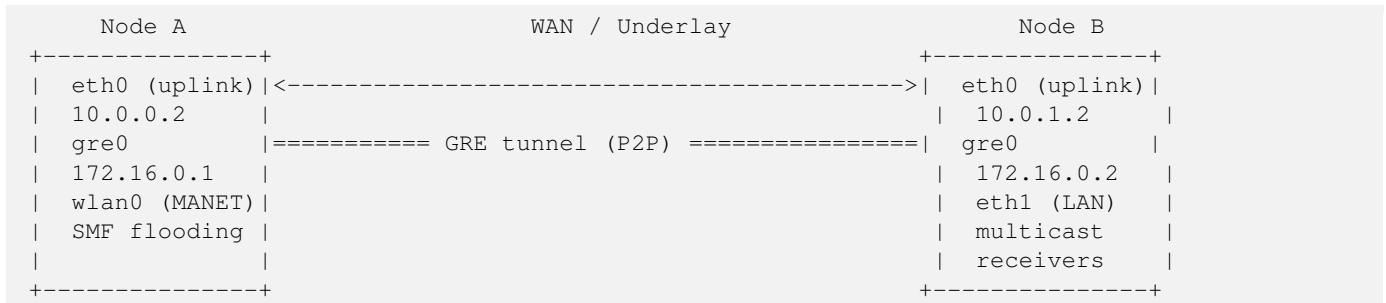
The other case that requires **map** is a "lightweight" or "metadata" GRE interface (externally-managed tunnel with no fixed endpoints on the device). nrlsmf logs a warning that the interface is missing tunnel endpoint addressing until **map** supplies it.

When Elastic Multicast is enabled, GRE tunnel endpoints are also used to identify the correct interface for elastic control messages (for example, `EM_ACK`). JSON configuration file support for GRE tunnel mapping is not yet available; use command-line or run-time remote control commands instead.

5.1.2 Point-to-Point GRE

A point-to-point GRE tunnel has one fixed local address and one fixed remote address, configured once at tunnel setup. There is no ambiguity about where an outbound packet goes, so no ongoing peer-resolution mechanism is needed.

Topology:



Kernel-level setup (Node A):

```

ip tunnel add gre0 mode gre local 10.0.0.2 remote 10.0.1.2 ttl 32
ip addr add 172.16.0.1/30 dev gre0
ip link set gre0 up

```

nrlnsmf configuration (Node A):

```

nrlnsmf resequence on \
    cf wlan0 \
    rpush wlan0,gre0 \
    push gre0,wlan0 \
    layered gre0

```

- `cf wlan0` — classical flooding within the MANET.
- `rpush wlan0,gre0` — relay MANET-originated traffic across the tunnel, tagging it for DPD.
- `push gre0,wlan0` — relay traffic from the far side back into the MANET (already tagged).
- `layered gre0` — a point-to-point tunnel gains nothing from being re-flooded out itself.

Typical use case: connecting two MANET "islands," or a MANET to a fixed-infrastructure multicast network, across a WAN hop with no native multicast support.

5.1.3 Multipoint GRE (mGRE)

A single mGRE interface represents multiple remote peers instead of one. GRE itself has no built-in signaling for which peer a packet goes to; three different mechanisms answer that question for overlay unicast, entirely below nrlnsmf.

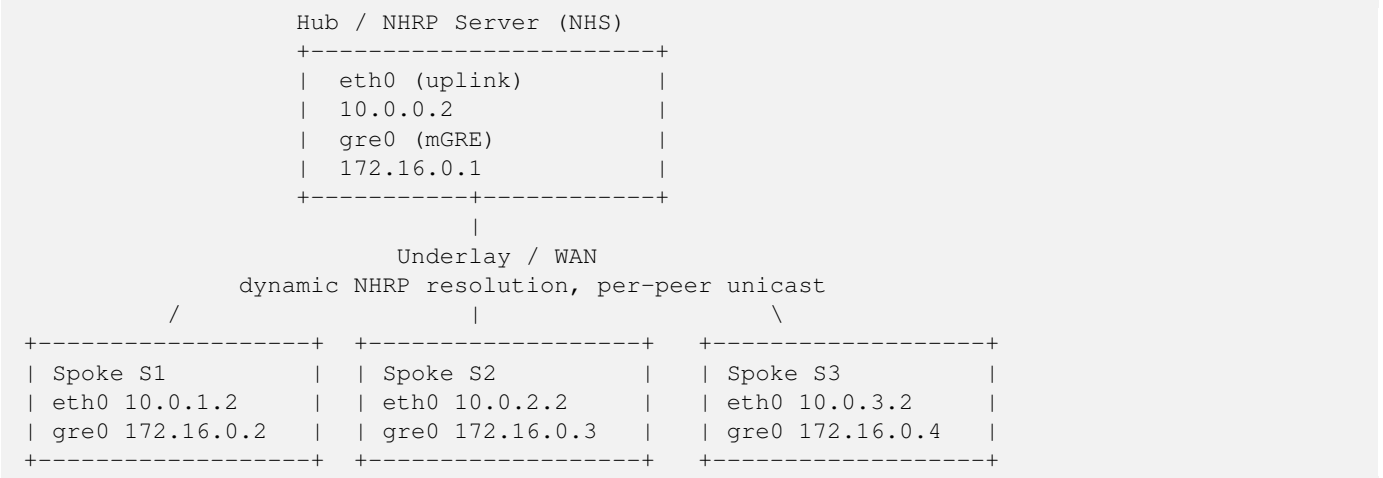
Modes A and B are, at the kernel level, the same mechanism — an mGRE interface with no fixed remote (`remote 0.0.0.0`) and a neighbor-resolution table that maps overlay tunnel addresses to underlay peer addresses — differing only in whether that table is maintained by a routing daemon or by hand. Mode C is fundamentally different: instead of per-peer unicast resolution, it turns the tunnel into a virtual broadcast/multicast segment.

5.1.3.1 Mode A — NHRP-resolved mGRE (dynamic hub-and-spoke)

Peers ("spokes") register their real underlay address with a Next Hop Resolution Protocol (NHRP) server (typically a hub, or "NHS" — Next Hop Server). Any peer needing to reach another resolves the destination's tunnel-address-to-underlay-address mapping dynamically through the NHS, and the kernel encapsulates directly to the resolved address. This is the mechanism behind Cisco DMVPN-style deployments; on Linux/FRR it is provided by FRR's `nhrpd` daemon (RFC 2332), which handles

NHRP registration and resolution independently of nrlsmf. nrlsmf itself has no NHRP support and does not need any: by the time a decapsulated packet reaches nrlsmf on the mGRE interface, nhrpd and the kernel have already resolved and delivered it.

Topology:



Kernel + FRR setup (Hub):

```
ip tunnel add gre0 mode gre local 10.0.0.2 key 42 ttl 64
ip addr add 172.16.0.1/32 dev gre0
ip link set gre0 up
```

Omitting remote is the same as remote 0.0.0.0 (kernel wildcard):

```
ip tunnel add gre0 mode gre local 10.0.0.2 remote 0.0.0.0 key 42 ttl 64
```

```
interface gre0
ip nhrp network-id 1
ip nhrp registration no-unique
```

The hub's own ordinary underlay address (10.0.0.2, whatever interface it already uses to reach the WAN) doubles as its NBMA identity — no separate loopback or special addressing is required.

Kernel + FRR setup (Spoke S1; S2 and S3 use 10.0.2.2/172.16.0.3 and 10.0.3.2/172.16.0.4):

```
ip tunnel add gre0 mode gre local 10.0.1.2 key 42 ttl 64
ip addr add 172.16.0.2/32 dev gre0
ip link set gre0 up
```

Equivalent wildcard remote:

```
ip tunnel add gre0 mode gre local 10.0.1.2 remote 0.0.0.0 key 42 ttl 64
```

```
interface gre0
ip nhrp network-id 1
ip nhrp nhs 172.16.0.1 nbma 10.0.0.2
ip nhrp registration no-unique
```

nhrpd installs overlay-to- underlay neighbors. A spoke's neighbor table is the NHS; the hub's table is every registered spoke. Overlay unicast therefore looks like static NBMA. Overlay multicast does not: a spoke has only the hub as a mapped unicast remote, so overlay multicast from a spoke goes to the hub, and the hub's nrlsmf must replicate it to the other spokes. Spokes are **layered** so a packet received on the tunnel is not flooded back out of it. The hub is *not* layered, because it is the overlay replicator.

nrlsmf configuration (hub; each spoke underlay address is a remote):

```
nrlsmf add overlay,cf,eth1,gre0 \
    map gre0,10.0.0.2,10.0.1.2 \
    map gre0,10.0.0.2,10.0.2.2 \
    map gre0,10.0.0.2,10.0.3.2
```

Equivalent using neighbor learning (nhrpd installs the same unicast remotes in `ip neigh`):

```
nrlsmf add overlay,cf,eth1,gre0 \
    map gre0,10.0.0.2,dynamic
```

nrlsmf configuration (spoke S1; S2 and S3 use their own underlay address):

```
nrlsmf add overlay,cf,eth1,gre0 \
    layered gre0 \
    map gre0,10.0.1.2,dynamic
```

`map . . . ,dynamic` is the usual inject config with NHRP: `nhrpd` programs `ip neigh`, and `nrlsmf` learns those unicast remotes. Explicit maps of `0.0.0.0` record the kernel wildcard remote for DPD / Elastic Multicast bookkeeping; they are not send destinations and not a synonym for `dynamic`.

Production DMVPN deployments typically add IPsec (`nhrpd` integrates with `strongSwan` via the VICI protocol) to protect the overlay traffic; that is independent of `nrlsmf` and omitted here. Spoke-to-spoke "shortcut" tunnels (DMVPN Phase 2/3) require additional NFLOG/iptables configuration and are not used for overlay multicast with `nrlsmf` classic flooding — see FRR's `nhrpd` documentation for that setup.

5.1.3.2 Mode B — Statically-mapped mGRE (manual NBMA table)

Mechanically identical to Mode A at the kernel level — same `remote 0.0.0.0` mGRE interface — but the peer-resolution table is populated by hand instead of by a routing daemon.

Overlay unicast is resolved by the kernel neighbor table (overlay-addr to underlay-addr). On Node A:

```
ip neigh add 172.16.0.2 lladdr 10.0.1.2 dev gre0 nud permanent
ip neigh add 172.16.0.3 lladdr 10.0.2.2 dev gre0 nud permanent
ip neigh add 172.16.0.4 lladdr 10.0.3.2 dev gre0 nud permanent
```

Overlay multicast has no entry in that table (and a single send cannot fan out to several unicast underlay destinations). `nrlsmf` injects overlay multicast once per mapped remote in its interface-info table. Those remotes are usually the same unicast peers as in `ip neigh`, but a mapped remote may also be an underlay multicast group (see Section 5.1.3.4).

The topology has the same shape as Mode A's diagram, with static `ip neigh` entries in place of `nhrpd`; typically used for a small, fixed set of known peers.

nrlsmf configuration (Node A, local underlay 10.0.0.2):

```
nrlsmf add overlay,cf,eth1,gre0 \
    layered gre0 \
    map gre0,10.0.0.2,10.0.1.2 \
    map gre0,10.0.0.2,10.0.2.2 \
    map gre0,10.0.0.2,10.0.3.2
```

Equivalent using neighbor learning from the `ip neigh` table above (`dynamic` dumps that table and keeps it updated via `NEWNEIGH/DELNEIGH`):

```
nrlsmf add overlay,cf,eth1,gre0 \
    layered gre0 \
    map gre0,10.0.0.2,dynamic
```

The wildcard remote can be mapped explicitly. This is allowed, but it is not a send destination and not a synonym for `dynamic`. Opening `gre0` already records it from the kernel, so it is usually redundant:

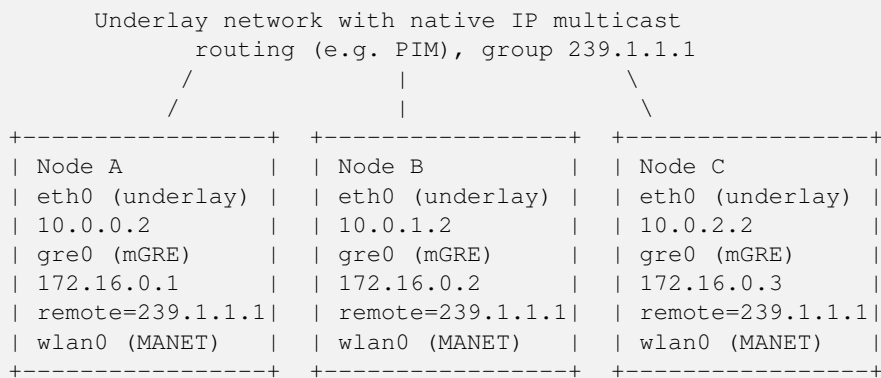
```
nrlsmf add overlay,cf,eth1,gre0 \
    layered gre0 \
    map gre0,10.0.0.2,0.0.0.0
```

Typical use case: a handful of fixed, stable sites where the operational overhead of running NHRP is not worth it, but a single mGRE interface (instead of one point-to-point interface per peer) is still convenient.

5.1.3.3 Mode C — Multicast-underlay mGRE

The tunnel's remote address is configured as an IP multicast group address rather than a unicast peer or a wildcard. A single encapsulated transmission is sent once, to that group; the underlay network's own multicast routing (for example, PIM) replicates and delivers it to every peer that has joined the group. This is the mode nrlsmf's **ujoin/uleave** commands exist for, and it is the most natural fit for SMF: symmetric, no hub, no per-peer replication — the same job SMF already does on a physical broadcast medium, relocated onto a WAN.

Topology:



Kernel-level setup (Node A; Node B and Node C use 10.0.1.2/172.16.0.2 and 10.0.2.2/172.16.0.3 — symmetric, no hub role):

```
ip tunnel add gre0 mode gre local 10.0.0.2 remote 239.1.1.1 ttl 16
ip addr add 172.16.0.1/24 dev gre0
ip link set gre0 up
```

nrlsmf configuration (each node):

```
nrlsmf add overlay,cf,gre0,wlan0 \
    layered gre0 \
    ujoin 239.1.1.1,eth0
```

ujoin/uleave are issued against the *underlay* interface (eth0), not the GRE interface itself — they join or leave the multicast group on the physical network so the kernel GRE device (whose remote is already that group) actually receives the encapsulated traffic to decapsulate. Mirror with `uleave 239.1.1.1,eth0` on shutdown or reconfiguration. This is not the same as mapping a multicast group onto a wildcard-remote mGRE device; the kernel already demuxes GRE-in-multicast onto the tunnel, so nrlsmf does not also strip GRE from underlay capture (that would double-process the packet).

When Elastic Multicast routing is enabled, the same tunnel endpoint information is used to route EM control traffic (for example, `EM_ACK`) to the correct interface — another reason accurate endpoint state matters even when data-plane decapsulation "just works."

This mode only works if the underlay genuinely supports IP multicast routing end-to-end. If it does not, GRE-over-multicast can silently fall back to resolving individual neighbors instead of true multicast fan-out — worth verifying the underlay's multicast path explicitly before relying on this mode.

5.1.3.4 Mixed underlay: multicast group plus unicast peers

Wildcard-remote mGRE (Modes A and B) can inject overlay multicast to an underlay multicast group as well as to unicast peers. That is useful when some sites can join the group and others cannot: one transmission covers the multicast-capable set, and additional transmissions cover the unicast-only peers. The kernel tunnel remote stays 0.0.0.0; this is not multicast-underlay mGRE (Mode C), whose device remote is already the group.

A Linux wildcard-remote mGRE device cannot send overlay multicast as GRE-in-multicast by itself (there is no multicast `sll_addr` for `PF_PACKET`), and it does not demux inbound GRE-in-multicast either: kernel GRE lookup matches the tunnel local address, not the outer multicast destination. Mapping the group as an inject remote makes nrlsmf send GRE-in-multicast once to that group. **ujoin** of the same group on the underlay interface both joins IGMP and enables an underlay capture so nrlsmf can strip GRE and treat the inner packet as inbound on the overlay GRE interface. Do not enable that capture on a multicast-underlay GRE device: the kernel already delivers the packet on the tunnel, and a second strip would double-process it.

Topology: five overlay routers on one wildcard-remote mGRE. Nodes A, B, and C can join underlay group 239.1.1.1. Nodes D and E cannot.

```
Node A -- lan0 --\
Node B -- lan1 ---\
Node C -- lan2 ---- underlay   A/B/C join 239.1.1.1
Node D -- lan3 ---/           D/E unicast only
Node E -- lan4 --/
```

Kernel-level setup (Node A; B–E use their own underlay and overlay addresses). Same wildcard-remote mGRE as Modes A and B — the tunnel remote is 0.0.0.0, not the underlay group:

```
ip tunnel add gre0 mode gre local 10.0.0.2 remote 0.0.0.0 ttl 64
ip addr add 172.16.0.1/24 dev gre0
ip link set gre0 multicast on
ip link set gre0 up
```

Overlay unicast still needs kernel neighbors (Node A; these can also be added by a dynamic protocol such as NHRP):

```
ip neigh add 172.16.0.2 lladdr 10.0.1.2 dev gre0 nud permanent
ip neigh add 172.16.0.3 lladdr 10.0.2.2 dev gre0 nud permanent
ip neigh add 172.16.0.4 lladdr 10.0.3.2 dev gre0 nud permanent
ip neigh add 172.16.0.5 lladdr 10.0.4.2 dev gre0 nud permanent
```

On A, B, and C, point the underlay group at the physical iface so GRE-in-multicast has an output device:

```
ip route replace 239.1.1.1/32 dev eth0
```

nrlsmf configuration (Node A; B and C are the same with their own underlay address):

```
nrlsmf add overlay,cf,eth1,gre0 \
    layered gre0 \
    ujoin 239.1.1.1,eth0 \
    map gre0,10.0.0.2,239.1.1.1 \
    map gre0,10.0.0.2,10.0.3.2 \
    map gre0,10.0.0.2,10.0.4.2
```

Overlay multicast inject from A is then three GRE transmissions: one to 239.1.1.1, one to D (10.0.3.2), and one to E (10.0.4.2). Overlay unicast still uses kernel `ip neigh`.

Nodes D and E create the same wildcard-remote tunnel (Node D: local 10.0.3.2 / overlay 172.16.0.4) and the same style of `ip neigh` entries. They do not add the underlay group route.

*nrlsmf configuration (Node D; E is the same with its own underlay address and no **ujoin**):*

```
nrlsmf add overlay,cf,eth1,gre0 \
    layered gre0 \
    map gre0,10.0.3.2,10.0.0.2 \
    map gre0,10.0.3.2,10.0.1.2 \
    map gre0,10.0.3.2,10.0.2.2 \
    map gre0,10.0.3.2,10.0.4.2
```

layered on the overlay GRE interface keeps a packet that arrived on the tunnel from being flooded back out of it, so a unicast-only router does not re-inject toward the multicast set. Traffic that arrives on `eth1` is still injected onto `gre0`.

5.1.4 External (metadata) GRE

A fourth way GRE encapsulation parameters can be resolved, distinct from all three mGRE modes above: `ip link add ... type gre external` creates an interface with no fixed local, remote, or key at all. Instead, whatever adds routes or flow rules for it supplies the encapsulation parameters per destination, using Linux's lightweight tunnel ("lwtunnel") route `encap`:

```
ip route add 172.16.0.2/32 encap ip id 42 src 10.0.0.2 dst 10.0.1.2 ttl 16 dev gre0
```

This is the mechanism SDN controllers and OVS/OVN typically use to build many per-flow or per-peer tunnels dynamically out of a single device. It is conceptually the multipoint-resolution equivalent of Mode B's static neighbor table — same job, expressed as routes instead of neighbor entries — but because the device itself has no fixed endpoint addresses, this is the one case where `nrlsmf`'s **map** command is actually mandatory: there is nothing for `nrlsmf` to auto-discover from the interface, and it will log a warning (GRE interface '`<name>`' missing tunnel endpoint addressing. (must map it)) until **map** supplies a local address. Overlay unicast still uses the kernel `lwtunnel` routes. Overlay multicast inject has no single remote, so `nrlsmf` also needs unicast remotes in that same map table (map `<iface>`, `<local>`, `<peer>` per peer). map `..., 0.0.0.0` is only the wildcard bookkeeping form; map `..., dynamic` does not apply (no `ip neigh`).

nrlsmf configuration (Node A):

```
nrlsmf add overlay,cf,gre0 \
    layered gre0 \
    map gre0,10.0.0.2,10.0.1.2 \
    map gre0,10.0.0.2,10.0.2.2
```

5.1.5 Where Each Mode Fits

Table 5: GRE / mGRE modes

Mode	Peer resolution	nrlsmf extras	Typical use
P2P GRE	Fixed at setup	typically layered	Two fixed sites; MANET-island bridging over a WAN hop
mGRE Mode A (NHRP)	Dynamic, per-peer unicast (FRR nhrpd)	map <code>..., dynamic</code> (or explicit unicast maps); spokes layered , hub not (replicator)	Hub-and-spoke overlay multicast; NHRP/DMVPN-style tooling already in place
mGRE Mode B (static NBMA)	Static, per-peer unicast (manual <code>ip neigh</code>)	explicit unicast maps (or map <code>..., dynamic</code>); typically layered	Small, fixed set of known peers; avoids running NHRP
mGRE mixed underlay	Same wildcard-remote device as Modes A/B; overlay multicast inject to an underlay group <i>and</i> unicast peers	ujoin plus map of the group (GRE-in-mcast send and underlay demux); unicast maps for peers that cannot join; typically layered	Some sites can join underlay multicast, others cannot
mGRE Mode C (multicast underlay)	None — underlay multicast fan-out (PIM)	ujoin/uleave on the underlay interface (IGMP join; kernel demuxes GRE-in-mcast onto the tunnel)	Symmetric SMF/MANET-gateway overlays
External (metadata) GRE	Per-destination <code>lwtunnel</code> routes (often SDN-managed)	map is mandatory; remotes for overlay multicast inject; typically layered	SDN/OVS-managed overlays; an external controller provisions tunnel endpoints

- Modes A and B are the same underlying mechanism — an mGRE interface with no fixed remote and a peer-resolution table — differing only in whether that table is maintained dynamically (NHRP) or by hand. The same wildcard-remote device can also map an underlay multicast group as one overlay-multicast inject dest (mixed underlay); that is not multicast-underlay mGRE (Mode C).
- Point-to-point and multicast-underlay mGRE (Mode C) rely on nrlsmf's automatic endpoint discovery (one configured remote). Modes A and B overlay unicast use kernel `neigh` (`nhrpd` or `ip neigh`); overlay multicast inject uses SMF's map table, filled by explicit **maps** (unicast peers and/or an underlay multicast group) and/or `map . . . , dynamic`.
- **map** is mandatory for "external"/metadata GRE (no endpoints on the device). For Mode A/B overlay multicast, mapped remotes (or `map . . . , dynamic`) are required. `0.0.0.0` means wildcard remote, matching the kernel. It is not needed for point-to-point or Mode C. Mapping an underlay multicast group onto a wildcard-remote mGRE also requires **ujoin** of that group so inbound GRE-in-multicast can be demuxed.
- In every mode, once packets reach nrlsmf they are handled by the same relay logic (`cf / smpr / ecds / push / rpush / merge / rmerge / layered`) as any other interface. Overlay multicast inject onto a wildcard-remote mGRE is the extra send-side step: one transmission per mapped remote (unicast peer or underlay group), still using that same relay decision. Overlay GRE interfaces are typically **layered**; the NHRP hub is not, because it replicates overlay multicast.

5.1.6 GRE Tunnel Commands

Table 6: GRE Tunnel Commands

<pre>map <iface>, <localAddr> [, <remoteAddr>]</pre>	Associate tunnel endpoint address information with <iface>. If <remoteAddr> is omitted or invalid, the <localAddr> is recorded as an ordinary local interface address. If <remoteAddr> is valid, the mapping is recorded as tunnel endpoint information for GRE or other tunnel interfaces. For mGRE, <remoteAddr> may be 0.0.0.0 (kernel wildcard remote, not a send dest). <code>dynamic</code> learns unicast remotes from the kernel neighbor table (NHRP or static NBMA). It does not apply to metadata/"external" GRE. Multiple commands with different remotes record multiple inject destinations (unicast peers and/or an underlay multicast group) on a wildcard-remote mGRE interface or a metadata GRE device.
<pre>unmap <iface>, <localAddr> [, <remoteAddr>]</pre>	Remove a previously configured tunnel or local address mapping from <iface>. The address arguments must match those supplied to "map". A remote of <code>dynamic</code> stops neighbor learning and removes learned remotes only.
<pre>ujoin <groupAddr>, <iface></pre>	Join multicast group <groupAddr> on underlay interface <iface> so the host receives GRE-in-multicast. On multicast-underlay mGRE this is an IGMP join so the kernel tunnel (remote already the group) can decapsulate. On a wildcard-remote mGRE device, if that same group is also a mapped inject remote, this additionally enables underlay capture so nrlsmf can strip GRE and treat the inner packet as inbound on the overlay GRE interface. The group address must be a valid IP multicast address.
<pre>uleave <groupAddr>, <iface></pre>	Leave multicast group <groupAddr> on underlay interface <iface>.

5.2 Virtual Interface Commands

SMF supports virtual interface constructs that associate a user-space forwarding path with one or more underlying physical interfaces. These are commonly used for host integration, composite devices, and encapsulation.

Table 7: Virtual Interface Commands

device <vifName>, <ifaceName> [/ {t r d}] [, <addr>...]	Create a virtual SMF interface (vif) associated with one or more physical interfaces. Optional flags on <ifaceName> are t (transmit), r (receive), or d (delete). Additional addresses may be listed after the interface name.
cid <vifName>, <iface1>[/ {t r d}] [, <iface2>...]	Add or remove elements of a Composite Interface Device associated with an SMF virtual interface.
layered <ifaceList>	Mark interfaces as layered, indicating they have their own underlying multicast distribution mechanism.
igmpProxy <ifaceList>	Send IGMP joins on listed interfaces for groups of interest. Typically used with layered interfaces.
encapsulate <ifaceList>	Use IPIP encapsulation for outbound unicast packets on listed SMF device interfaces.
filterDups {on off}	Enable or disable duplicate packet filtering for SMF device operation. (default = on)
route <dstAddr>, <nextHopAddr>	Add a debug route used with encapsulation testing.

5.3 VRF and FRR Integration

nrlsmf can associate interfaces with named virtual routing and forwarding (VRF) contexts. When run alongside FRR, VRF information may be imported at startup.

Table 8: VRF Commands

vrf <vrf-name>, [<vrf-id>] <ifaceList>	Associate one or more interfaces with the named VRF. An optional numeric VRF identifier may be supplied.
with-frr	Run alongside FRR and pull configuration where appropriate, such as VRF information. This option must appear early on the command line.

5.4 JSON Configuration

nrlsmf supports loading and saving configuration in JSON format. Some advanced options (including GRE tunnel mapping) are not yet available via JSON and must be configured using command-line or remote control commands.

Table 9: Configuration Commands

load <configFile>	Load an nrlsmf JSON configuration file at startup or at run-time.
save <configFile>	Save the current configuration to a JSON file, typically upon exit.
stats	Return interface and group information via the remote control interface.

The "Remote-only Commands" listed here can be invoked only via the run-time, interprocess remote-control interface (see [Remote Control Interface](#)). The principal reason these commands are available only via inter-process communication is that the commands have a partially binary format to convey lists of MAC addresses to control aspects of nrlsmf operation. In each of

these commands, an ASCII prefix is given to indicate the command type, followed by an ASCII "space" character, and then the binary list of concatenated 6-byte Ethernet MAC addresses (in Big Endian order). The length of the datagram command sent to nrlsmf implies the length of the MAC address list. The "selectorMac" and "neighborMac" apply to control of S-MPR forwarding. The "mneBlockMac" command is provided to support proper SMF behavior in the **NRL Mobile Network Emulation (MNE)** environment.

Table 10: Remote-only Commands

<pre>selectorMac <binary macAddrArray></pre>	This command can be used by external processes (e.g., nrlolsrd) to control nrlsmf S-MPR forwarding. This command sets the list of MAC addresses that have selected the local nrlsmf node as a Multi-Point Relay (MPR). The format of the <binary macAddrArray> is a list of 6-byte Ether-type MAC addresses. Thus, the length of this "array" is a multiple of 6 bytes. An ASCII "space" character delimits the literal "selectorMac" command string from the binary array of MAC addresses.
<pre>neighborMac <binary macAddrArray></pre>	This command can be used by external processes (e.g., nrlolsrd) to control nrlsmf S-MPR forwarding. This command sets the list of MAC addresses that have been identified as symmetric, 1-hop neighbors of the local nrlsmf node. The format of the <binary macAddrArray> is a list of 6-byte Ether-type MAC addresses. Thus, the length of this "array" is a multiple of 6 bytes. An ASCII "space" character delimits the literal "neighborMac" command string from the binary array of MAC addresses.
<pre>mneBlockMac <binary macAddrArray></pre>	This command enables the nrlsmf process to be compatible with the NRL Mobile Network Emulator (MNE) system that uses MAC-based blocking to emulate the connectivity of mobile network topologies. The command sets the list of addresses that should be blocked (i.e., are not reachable as 1-hop neighbors) of the local nrlsmf node. The NRL MNE processes use this command as needed. This command is only available when nrlsmf is compiled with the MNE_SUPPORT macro enabled.
<pre>smfPkt <binary packet info></pre>	This interprocess message is generated by nrlsmf and sent via "remote control interface" interprocess communication to the process identified by the "tap" command, if applicable. Additionally, this same message can be sent to nrlsmf to pass packets back to nrlsmf for forwarding. The format of the <binary packet info> is described below.

5.5 The "smfPkt" Command Format

The "smfPkt" command is used to pass packets to/from nrlsmf and external processes. This allows for external processes to insert themselves into the SMF forwarding process and provide additional filtering, etc of packets that SMF would forward. The "argument" of the "smfPkt" command is actually a binary format that includes a header with forwarding information and an entire Ether-type frame structure (i.e., 14-byte Ether-type header and IP packet payload). Since the "smfPkt" string and its delimiting ASCII "space" character is a total of 7 bytes in length, the binary content begins at the 8th byte of the "smfPkt" command message.

The binary content begins with 1 byte that is an "indexCount", that indicates the length of a list of interface indices that immediately follows. Each interface index is one byte. The first interface index in the list identifies the network interface upon which the subject packet arrived (i.e., the "inbound interface index"). The remaining interface indices in the list (the "outbound interface index list") are the interfaces on which the subject packet will be (or would have been) forwarded by the nrlsmf process. Again, note that the "indexCount" is one greater than the number of outbound interface indices (i.e., the count includes the inbound interface index).


```
nrslsmf --cli -i smf-rl -c "show groups details"
Flags: A = Active, K = Ack
Selected: Y = current upstream, N = additional inbound
Fwd: B = Block, H = Hybrid, L = Limit, F = Forward, D = Deny
Why: managed = IGMP/static last hop, ack = EM_ACK, - = default
```

group	saddr	flags	ack	recv	pps
239.0.0.1	192.168.55.2	AK	yes	41	8.8
sources					
iif	upstream	sel			
gre1	10.0.0.2	Y			
downstream					
oif	mode	why	fwd	sent	pps
eth1	elastic	managed	F	41	8.8
gre1	elastic	ack	F	0	0.0

show groups memberships is the controller table: S static (join), M managed (IGMP via with-frr), E elastic (downstream EM_ACK). This is not the last-hop skip list.

```
nrslsmf --cli -i smf-rl -c "show groups memberships"
Flags: S = Static, M = Managed, E = Elastic
```

group	saddr	iface	flags	relays
239.0.0.1	*	eth1	M	-
239.0.0.1	192.168.55.2	gre1	E	10.0.0.2

show igmp groups is the last-hop list filled by with-frr. A managed host interface that does not list the group is not used as a last hop.

```
nrslsmf --cli -i smf-rl -c "show igmp groups"
Flags: M = Managed (last-hop HasActiveMembership)
```

iface	flags	groups
eth1	M	239.0.0.1
gre1	-	-

7 Current Limitations

There are limitations in the use of some nrslsmf options. Many of these limitations are a result of nrslsmf being a cross-platform, user-space implementation. Many of these subtleties could be overcome with a kernel implementation of the code.

For example, to take advantage of configured firewall rules and/or quality-of-service (QoS) policies for multicast data transmission and forwarding, it may be desirable to use the nrslsmf "firewallForward" and/or "firewallCapture" options. There are some limitations associated with the user-space mechanisms available to capture and "forward" packets in this manner.

1. The "rpush" and "rmerge" commands must be used very carefully when used in combination with the "firewallCapture on" option:
 - The "firewallCapture" option doesn't necessarily provide the source MAC address of received packets properly:
 - On Linux, locally generated packets have some random source MAC address from the 'ip_queue' capture mechanism (thus can't detect it is receiving packets it previously sent and the resequencing bypasses DPD and a packet "cyclone" to TTL=0 results)
 - On BSD/macOS, ProtoDetour doesn't get the source MAC address at all for the "firewallCapture" mode ("layer 2" firewall rule would be needed).
 - So only use "firewallCapture on" when absolutely necessary.

2. For IPv4 resequencing, the ID value of ZERO is avoided since some operating systems (e.g., BSD) will automatically re-ID packets that have an ID value of ZERO when "firewallForward on" is used.
3. If large multicast packets are sent by hosts that require IP fragmentation, "firewallCapture on" must be used for SMF forwarding to work (SMF duplicate packet detection doesn't like fragments and the default Ethernet frame capture mode gets individual fragments while the "firewallCapture on" mode gets fully re-assembled IP packets).
4. IPv6 operation is more limited and creates a challenge to apply queuing rules or traffic shaping to forwarded IPv6 traffic:
 - IPv6 raw sockets don't allow full control of IP packet header as IPv4 raw sockets do.
 - Thus the current "firewallForward" using raw socket for forward doesn't work.
 - BSD firewall "divert" option evidently work with IPv6 evidently (so no BSD/MacOS "firewallCapture" or "firewallForward")

Some of these limitations may be addressed in future versions of nrlsmf if possible. It may be possible in some cases that system configuration (perhaps using virtual interface capabilities) may provide work-around solutions for some of these issues.

8 Future Plans

There are a number of additional features and refinements planned for the nrlsmf implementation. Some of these (in roughly priority order) include:

1. Built-in support for the Neighborhood Discovery Protocol (NHDP)
2. Option to apply "taggerID" field as described in SMF Internet Draft.
3. Option to load a "config" file for complex configurations
4. Replication of intercepted outbound locally-generated multicast packets to multiple interfaces (i.e., instead of the usual host transmission of multicast on a single specified interface.
5. Filters for applying per-group (destination multicast address) forwarding rules (i.e., to support use of "nrlsmf" as a forwarding engine for experimental forms of group-specific multicast routing).
6. Alternative packet capture/forwarding mechanisms (e.g., virtual interface mechanisms).